

关键点：TIM2 挂在 APB1 上；当 APB1 的分频系数 不为 1 时，送到定时器的时钟是 APB1 的 2 倍： $f_{TIM2} = 2 \times 36 \text{ MHz} = 72 \text{ MHz}$ 。

这个 72 MHz 就是后面计算 PSC、ARR 所用的定时器时钟（以 CubeMX 时钟树显示的“APB1 Timer clocks”为准）。

3. 第 1 题：GPIO

- CubeMX 里把 PC13 配置为 推挽输出（Output Push Pull）；
- F103 蓝板板载 LED 接在 PC13，低电平点亮；
- 在 Tasks 的初始化 MainInit() 里把它写成低电平（不在 while(1) 里翻转）。

```
void MainInit(void)
{
    /* 低电平点亮板载 LED */
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
    ...
}
```

现象：板载 LED 常亮（见附录 图 1）。

4. 第 2 题：定时器 1 ms 更新中断与 tick

4.1 PSC / ARR 计算过程

TIM2 的更新中断周期：

$$T_{up} = (PSC + 1) \times (ARR + 1) / f_{TIM2}$$

要求 $T_{up} = 1 \text{ ms}$ ， $f_{TIM2} = 72 \text{ MHz}$ ，则：

$$(PSC + 1)(ARR + 1) = f_{TIM2} \times T_{up} = 72\,000\,000 \times 0.001 = 72000$$

取整数分频 $PSC + 1 = 72$ ，则：

$$PSC = 71, ARR + 1 = 72000 / 72 = 1000 \Rightarrow ARR = 999$$

代入验算：

$$T_{up} = (71 + 1) \times (999 + 1) / 72\,000\,000 = 72000 / 72\,000\,000 = 0.001 \text{ s} = 1 \text{ ms} \quad \checkmark$$

CubeMX 中设置：TIM2 → Prescaler = 71, Counter Period = 999, 并在 NVIC 里使能 TIM2 global interrupt。

4.2 全局 tick

按题目要求，变量名必须叫 tick，类型为全局 volatile uint32_t：

```
extern "C" {  
    volatile uint32_t tick = 0;  
}
```

volatile 防止编译器把自增优化掉；用 extern "C" 保证导出符号名就是 tick，Ozone 里可直接按名字监视。

4.3 Tasks 里的定时器更新回调（全工程只写这一份）

```
extern "C" void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)  
{  
    if (htim->Instance == TIM2) {  
        tick = tick + 1;          /* 每 1 ms 加 1 → 约 1000/s */  
  
        /* 第 2 题：喂狗（第 3 题时注释掉） */  
        HAL_IWDG_Refresh(&hiwdg);  
    }  
}
```

用 C++ 写回调必须加 extern "C"，否则覆盖不了 HAL 的弱符号；回调只保留这一份；没有用 HAL_Delay 或空循环去加 tick。

4.4 启动定时器

```
void MainInit(void)  
{  
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);  
    HAL_TIM_Base_Start_IT(&htim2); /* 启动 1ms 更新中断 */  
}
```

现象：Ozone 运行态下，tick 每秒约增加 1000（见附录 图 2）。

5. 第 3 题：看门狗 IWDG

5.1 超时计算

IWDG 由 LSI（内部低速时钟）驱动，F103 的 LSI 约 40 kHz。超时时间：

$$T_{out} = (Reload + 1) \times \text{分频系数} / f_{LSI}$$

取分频系数 = 64、Reload = 1249：

$$T_{out} = (1249 + 1) \times 64 / 40\ 000 = 80\ 000 / 40\ 000 = 2.0\ s$$

CubeMX 设置：Prescaler = IWDG_PRESCALER_64, Reload = 1249, 超时约 2 s。

5.2 现象与验证

- 第 2 题中，回调每 1 ms 调用一次 HAL_IWDG_Refresh(&hiwdg) 喂狗，tick 持续增长；
- 第 3 题 去掉回调里的 HAL_IWDG_Refresh，重新编译下载后：芯片约 2 s 得不到喂狗 → IWDG 溢出 → MCU 复位；复位后启动代码把全局变量清零，所以 tick 从 0 涨到约 2000，然后回到 0，不断重复；
- 用 Ozone 观察时让程序运行、不要停在断点上（CPU 停住时看门狗仍在计数，会掉线）。

现象：tick 呈锯齿状，涨到约 2000 后归零、循环（见附录 图 3）。

6. 三题与配置小结

题	外设/变量	配置	现象
1 GPIO	PC13	推挽输出，写低电平	板载 LED 常亮
2 定时器	TIM2	PSC=71, ARR=999 → 1 ms；回调 tick++ + 喂狗	tick 约 1000/s
3 看门狗	IWDG	分频 64, Reload 1249 → 2 s；回调去掉喂狗	tick 涨到约 2000 后归零

附录（图）

说明文档只在附录放图。

图 1（GPIO）：板载 LED 点亮



图 1 板载 LED 常亮

图 2（定时器）：tick 约每秒增加 1000

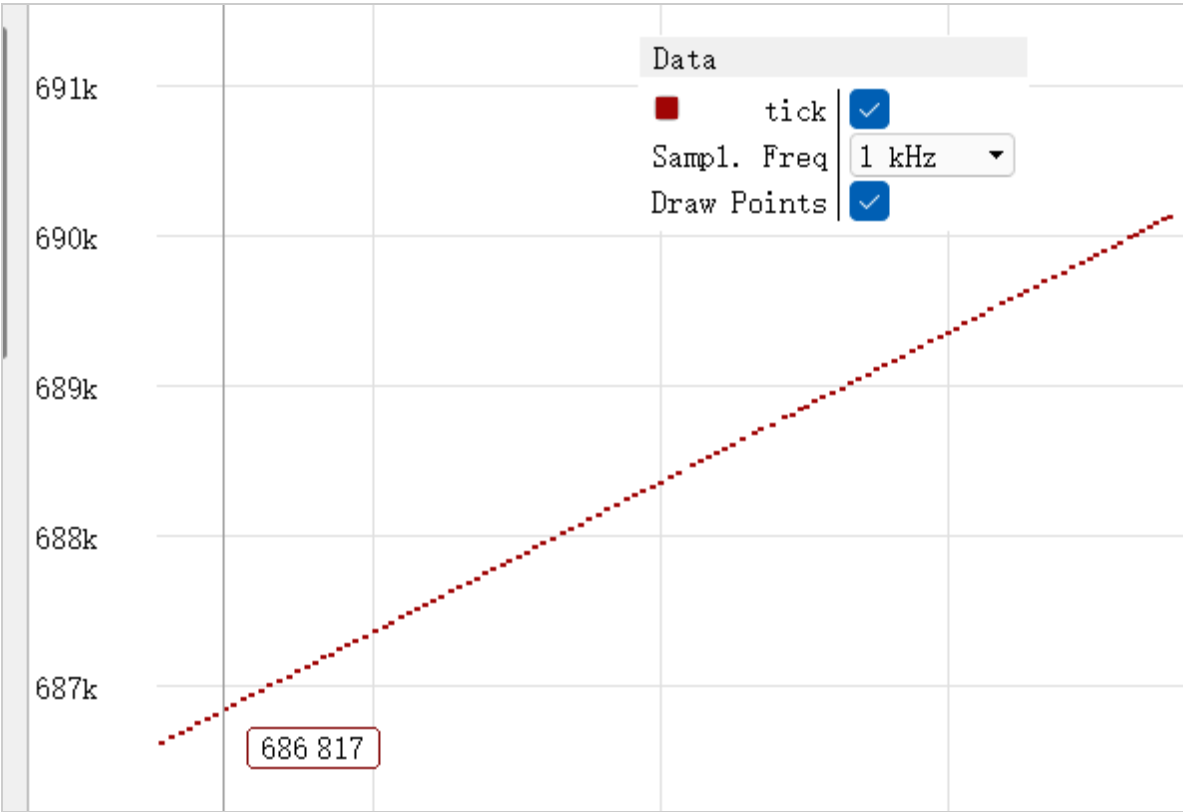


图 2-1 tick 采样数据 (Time 与 tick)

Data Sampling			Sampling Freq: 1 kHz
Index	Time	tick	
33	0.991 000	683 269	
34	1.021 000	683 298	
35	1.051 000	683 329	
36	1.079 000	683 356	
37	1.114 000	683 392	
38	1.156 000	683 434	
39	1.184 000	683 461	
40	1.212 000	683 489	
41	1.234 000	683 512	
42	1.265 000	683 543	
43	1.290 000	683 567	
44	1.317 000	683 594	
45	1.344 000	683 621	
46	1.380 000	683 658	
47	1.408 000	683 686	
48	1.442 000	683 720	
49	1.469 000	683 747	
50	1.506 000	683 784	
51	1.537 000	683 814	
52	1.570 000	683 848	
53	1.602 000	683 879	
54	1.633 000	683 911	
55	1.664 000	683 943	
56	1.700 000	683 978	
57	1.728 000	684 006	
58	1.759 000	684 037	
59	1.787 000	684 065	
60	1.820 000	684 098	
61	1.852 000	684 130	
62	1.883 000	684 161	
63	1.919 000	684 196	
64	1.946 000	684 224	
65	1.978 000	684 255	
66	2.005 000	684 283	
67	2.036 000	684 314	
68	2.064 000	684 342	
69	2.095 000	684 373	
70	2.133 000	684 410	
71	2.160 000	684 438	
72	2.195 000	684 473	
73	2.219 000	684 496	
74	2.250 000	684 527	
75	2.273 000	684 551	

图 2-2 tick 随时间上升的曲线（约 1000/s）

图 3（看门狗）：tick 涨到约 2000 后归零

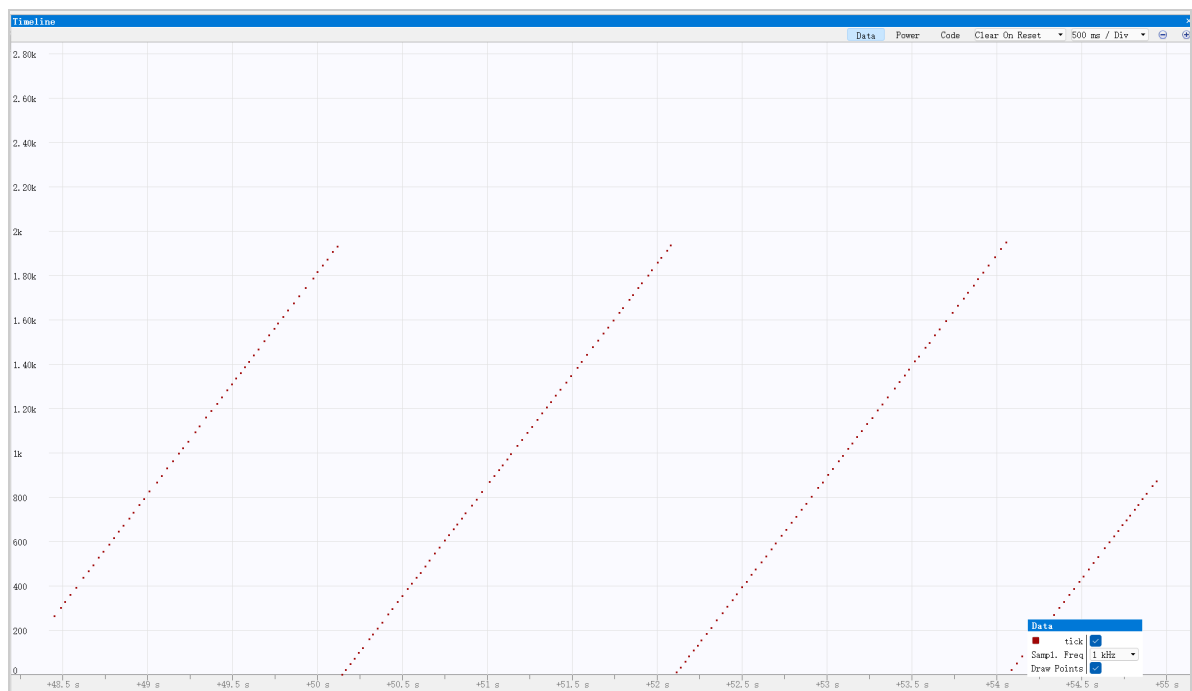


图 3 tick 锯齿状：涨到约 2000 后归零